

TrabFlow Technologies

# Especificación — Suite de Pruebas Automatizadas del Motor de IA de Presupuestos

Handoff para Claude Code · Junio 2026

## 1. OBJETIVO

Construir un script de pruebas 100% automático que valide el comportamiento del motor de IA de presupuestos (edge function trade-voice-to-quote) frente a 400 peticiones de trabajo realistas: 20 oficios de la Base Maestra × 20 peticiones típicas por oficio.

El objetivo no es solo comprobar que la función responde, sino clasificar cada respuesta según el comportamiento real del motor: partida encontrada en catálogo, partida sugerida por IA pendiente de precio, fallback a búsqueda web, respuesta vacía o error técnico.

La prueba debe poder relanzarse en cualquier momento (tras un despliegue, un cambio de prompt, una ampliación de catálogo) sin intervención manual, y producir siempre el mismo formato de informe comparable entre ejecuciones.

## 2. ENTRADA A LA FUNCIÓN — INVOCACIÓN POR TEXTO

La función acepta un campo de texto directo (transcript/descripcion), por lo que el script puede invocar la función sin pasar por grabación de audio ni Whisper. Esto es lo que hace viable la automatización completa.

```
POST {SUPABASE_URL}/functions/v1/trade-voice-to-quote
Headers:
  Authorization: Bearer {SERVICE_ROLE_KEY o JWT de test}
  Content-Type: application/json
Body:
{
  "transcript": "<texto de la petición>",
  "orgId": "<org_id de la organización de pruebas>",
  "skipTranscription": true
}
```

*Nota para Claude Code: confirmar el nombre exacto del campo de texto en el código actual de la función (transcript vs descripcion vs texto) antes de implementar el runner, y confirmar si existe ya un flag para saltar Whisper o si hay que añadirlo.*

## 3. ARQUITECTURA DEL SCRIPT

### 3.1 Estructura de carpetas

```
scripts/test-motor-ia/
├── queries.json           // 400 peticiones (20 oficios × 20)
├── run-tests.ts           // runner principal (Node/Deno)
└── classify.ts            // lógica de clasificación de resultado
```

```

├─ report-excel.ts      // genera test_results.xlsx
├─ report-word.ts      // genera test_summary.docx
├─ output/
│   └─ test_results.xlsx
│   └─ test_summary.docx
└─ raw/{oficio}_{n}.json // respuesta cruda por petición, para depurar

```

### 3.2 Flujo del runner

1. Carga queries.json (400 entradas: oficio, texto, id).
2. Crea/reutiliza una organización de pruebas (org\_id fijo) con su Base Maestra cargada — para que los resultados sean comparables entre ejecuciones.
3. Para cada query: invoca la función con reintento (máx. 2 reintentos en error 5xx/timeout), mide latencia, guarda la respuesta cruda en output/raw/.
4. Ejecuta las llamadas en lotes de 5-10 en paralelo (no las 400 a la vez, para no saturar la función ni el rate limit de Voyage/Anthropic).
5. Clasifica cada respuesta (ver sección 4).
6. Genera el Excel y el Word de resumen al finalizar todo el lote.
7. Si una query falla tras los reintentos, se marca como ERROR\_TECNICO y el runner continúa con la siguiente (nunca debe abortar el lote completo).

## 4. CLASIFICACIÓN DE CADA RESPUESTA

Cada una de las 400 respuestas debe quedar clasificada en una de estas categorías, en función del array partidas devuelto y de su campo estado:

| Categoría       | Condición                                                | Qué significa                                                                       |
|-----------------|----------------------------------------------------------|-------------------------------------------------------------------------------------|
| OK_CATALOGO     | Todas las partidas tienen estado: 'catalogo'             | El motor encontró todo en la Base Maestra. Caso ideal.                              |
| OK_MIXTO        | Mezcla de 'catalogo' y 'nueva_partida_sugerida'          | Parte encontrada, parte nueva — esperable y aceptable.                              |
| SOLO_SUGERIDAS  | 100% de las partidas son 'nueva_partida_sugerida'        | El catálogo no cubre nada de esa petición. Señal de hueco en Base Maestra.          |
| VACIO           | partidas: [] o ausente                                   | Fallo del motor — viola la regla 'nunca devuelvas partidas vacías'. Bug a reportar. |
| ERROR_TECNICO   | HTTP != 200, timeout, JSON inválido                      | Fallo de infraestructura/edge function, no de calidad de IA.                        |
| PRECIO_INVALIDO | Alguna partida 'catalogo' con precio_unitario = 0 o nulo | Violación de la regla de negocio: catálogo siempre debe traer precio.               |

Para cada respuesta también se registra: oficio\_detectado (¿coincide con el oficio esperado?), nº de partidas devueltas, tiempo de respuesta en ms, y si el plan\_trabajo es coherente con el texto de entrada (esto último puede marcarse para revisión manual, no automatizable con fiabilidad).

## 5. LAS 400 CONSULTAS DE PRUEBA

A continuación se listan los 20 oficios con 3 ejemplos de petición real de instalador cada uno, en el tono y nivel de detalle que debe tener todo el set. Claude Code debe completar cada

oficio hasta 20 peticiones siguiendo este mismo patrón: lenguaje de campo (no técnico-comercial), con medidas/cantidades cuando aplique, incluyendo variantes de "cambiar X por Y", "reparar", "instalar nuevo", "mantenimiento" y alguna petición ambigua o mal formulada a propósito (para probar el fallback).

### **1. Fontanería**

- Cambiar bañera por plato de ducha de 170x70 con grifería
- Reparar fuga en tubería de cobre bajo fregadero de cocina
- Instalar termo eléctrico de 100 litros en sustitución del actual

### **2. Electricidad**

- Cambiar cuadro eléctrico antiguo por uno nuevo con diferenciales
- Instalar 6 puntos de luz LED empotrados en salón
- Sustituir mecanismos de enchufes y interruptores en vivienda de 90 m<sup>2</sup>

### **3. Climatización/HVAC**

- Instalar split de aire acondicionado 3000 frigorías en dormitorio
- Revisar y recargar gas de equipo de aire acondicionado
- Sustituir caldera de gas por caldera de condensación

### **4. Calefacción**

- Instalar suelo radiante en cocina de 12 m<sup>2</sup>
- Cambiar radiadores antiguos por radiadores de aluminio
- Purgar y revisar instalación de calefacción central

### **5. Albañilería**

- Tabicar habitación de 10 m<sup>2</sup> con ladrillo y enlucido
- Picar y alicatar baño completo de 6 m<sup>2</sup>
- Reparar grieta en fachada y repintar zona afectada

### **6. Carpintería/Ventanas**

- Cambiar ventana de aluminio por una de PVC con doble cristal
- Instalar puerta de paso lacada en blanco
- Montar armario empotrado a medida en dormitorio

### **7. Cerrajería**

- Cambiar cerradura de seguridad en puerta de entrada
- Reparar persiana metálica que no sube
- Instalar reja de protección en ventana de planta baja

### **8. Pintura**

- Pintar salón y dos dormitorios de piso de 80 m<sup>2</sup>
- Lijar y barnizar puerta de madera exterior
- Aplicar gotelé y repintar techo de cocina

### **9. Suelos/Tarimas**

- Instalar tarima flotante en salón de 25 m<sup>2</sup>
- Sustituir suelo de terrazo por porcelánico en cocina
- Reparar baldosas levantadas en terraza

#### **10. Pladur/Escayola**

- Hacer falso techo de pladur en pasillo con focos empotrados
- Tabique de pladur para dividir habitación en dos
- Reparar grieta en techo de escayola

#### **11. Cristalería**

- Cambiar cristal roto de ventana doble acristalamiento
- Instalar mampara de ducha de cristal templado
- Sustituir espejo de baño por uno con luz LED integrada

#### **12. Persianas/Cierres**

- Motorizar persiana enrollable de salón
- Cambiar cinta y cajón de persiana en dormitorio
- Instalar toldo enrollable en terraza de 4 metros

#### **13. Jardinería**

- Instalar sistema de riego automático en jardín de 100 m<sup>2</sup>
- Podar seto y mantenimiento de césped mensual
- Colocar césped artificial en patio de 30 m<sup>2</sup>

#### **14. Telecomunicaciones**

- Instalar punto de red y wifi en oficina de 3 salas
- Tender cableado de fibra óptica hasta router
- Configurar repetidor wifi para cobertura en planta superior

#### **15. CCTV/Seguridad**

- Instalar 4 cámaras de videovigilancia exteriores con grabador
- Sustituir alarma antigua por sistema conectado a app móvil
- Revisar y reparar cámara de seguridad que no graba

#### **16. Energía Solar**

- Instalar placas solares en tejado para autoconsumo de vivienda unifamiliar
- Sustituir inversor de instalación fotovoltaica averiado
- Ampliar instalación solar existente con baterías de almacenamiento

#### **17. Ascensores/Elevadores**

- Revisión de mantenimiento mensual de ascensor en comunidad
- Reparar puerta de ascensor que no cierra bien
- Sustituir cable de tracción de ascensor

#### **18. Taller Mecánico**

- Cambiar pastillas de freno y discos en furgoneta de empresa
- Revisión y cambio de aceite y filtros de vehículo
- Reparar avería eléctrica en cuadro de mandos de coche

## 19. Limpieza Industrial

- Limpieza a fondo de nave industrial de 500 m<sup>2</sup>
- Limpieza de cristales en fachada de oficina de 3 plantas
- Desinfección y limpieza tras obra en local comercial

## 20. Mano de Obra General/Reformas

- Reforma integral de baño de 5 m<sup>2</sup> llave en mano
- Reforma de cocina completa con cambio de muebles e instalaciones
- Trabajo de mantenimiento general en local comercial: varias pequeñas reparaciones

## 6. FORMATO DE SALIDA — EXCEL

Una fila por consulta (400 filas), columnas:

| Columna            | Contenido                                        |
|--------------------|--------------------------------------------------|
| id                 | Identificador secuencial 1-400                   |
| oficio             | Oficio esperado (de queries.json)                |
| texto_consulta     | Texto enviado a la función                       |
| oficio_detectado   | Campo devuelto por la IA                         |
| coincide_oficio    | SI/NO                                            |
| categoria          | Una de las 6 categorías de la sección 4          |
| n_partidas         | Número de partidas devueltas                     |
| n_catalogo         | Nº de partidas con estado catalogo               |
| n_sugeridas        | Nº de partidas con estado nueva_partida_sugerida |
| latencia_ms        | Tiempo de respuesta                              |
| http_status        | Código HTTP                                      |
| error_detalle      | Mensaje de error si aplica                       |
| respuesta_raw_path | Ruta al JSON crudo en output/raw/                |

Incluir una segunda hoja "Resumen" con tabla dinámica: nº y % de cada categoría por oficio, y una tercera hoja "Errores" con solo las filas VACIO y ERROR\_TECNICO para revisión prioritaria.

## 7. FORMATO DE SALIDA — WORD

Documento de resumen ejecutivo (no las 400 filas), con:

- Cabecera con fecha de ejecución, nº total de consultas, % de éxito global (OK\_CATALOGO + OK\_MIXTO).

- Tabla por oficio: % OK\_CATALOGO, % OK\_MIXTO, % SOLO\_SUGERIDAS, % VACIO, % ERROR\_TECNICO.
- Lista de los oficios con peor tasa de cobertura de catálogo (candidatos a ampliar Base Maestra).
- Lista de las consultas que dieron VACIO o ERROR\_TECNICO, con el texto exacto, para reproducir el bug.
- Comparativa con la ejecución anterior si existe un test\_summary.docx previo (delta de % de éxito), para medir progreso entre despliegues.

## 8. ESTIMACIÓN DE TIEMPO

| Tarea                                                         | Tiempo estimado |
|---------------------------------------------------------------|-----------------|
| Confirmar contrato de entrada (texto) de trade-voice-to-quote | 0.5 h           |
| Crear org_id de pruebas + cargar Base Maestra de referencia   | 0.5 h           |
| Redactar queries.json completo (400 peticiones)               | 1.5 h           |
| Runner (llamadas + reintentos + guardado raw)                 | 1.5 h           |
| Lógica de clasificación                                       | 1 h             |
| Generador Excel (3 hojas)                                     | 1 h             |
| Generador Word resumen                                        | 1 h             |
| Pruebas end-to-end + ajustes                                  | 1.5 h           |

**Total estimado: 9 horas.**

## 9. CRITERIOS DE ACEPTACIÓN

8. El script se ejecuta con un único comando (p. ej. npm run test:motor-ia) sin intervención manual.
9. Genera siempre test\_results.xlsx y test\_summary.docx en output/, sobrescribiendo la ejecución anterior tras copiarla a output/history/.
10. Ninguna consulta individual que falle detiene el resto del lote.
11. El % de éxito global y por oficio es visible de un vistazo en el Word, sin necesidad de abrir el Excel.
12. Las 400 consultas y sus categorías quedan trazables y reproducibles (mismo queries.json → mismas categorías, salvo cambios reales en el motor o el catálogo).